TP Unidad 7 — NoSQL y nuevas formas de almacenamiento: Food Store

Bases de Datos II · Integrantes: Matias Limina - Nicolas Monjelardi - Lautaro Aguero · Repositorio: `github.com/nicolasmonjelardi/BaseDeDatos2-TP7-NoSQL` · 01/11/2026

Entorno: PostgreSQL 17.11 (base `foodstore` del proyecto integrador) y MongoDB 9.0.4 con mongosh 2.14.1, en local. Archivos adjuntos: `consultas_mongodb.js` y `consultas_postgresql.sql`. El resto de los scripts, los tests y las evidencias completas están en el repositorio.

1. Modelo documental y migración (Consignas 1 a 3)

Tablas y claves que intervienen en PostgreSQL

Tabla	Clave primaria	Claves foráneas y únicas	Restricciones relevantes
`pedido`	`id`	`cliente_id → cliente`	`estado` (enum, agregado en este TP), `forma_pago` (enum)
`pedido_detalle`	`(pedido_id, producto_id)`	`pedido_id → pedido`, `producto_id → producto`	`cantidad > 0`, `precio_unitario >= 0`, `subtotal >= 0`
`cliente`	`id`	`email` UNIQUE	—
`producto` / `categoria`	`id`	`categoria_id → categoria`	`precio >= 0`, `stock >= 0`

`pedido_detalle` es la tabla intermedia de la relación N:M y guarda el **precio unitario histórico**. `pedido` no tiene columna de total: el total es la suma de los subtotales.

Estado agregado. La consigna pide un estado por pedido, y la tabla `pedido` del equipo no lo tenía. Antes de agregarlo se tomó un respaldo con el script del TP6. Después se agregó `estado_pedido_enum`, de tipo `NOT NULL DEFAULT 'PENDIENTE'`, con el script idempotente `sql/00_estado_pedido.sql`. Los pedidos existentes son datos sintéticos de los TP 3 y 4: recibieron un estado por una regla fija, según la fecha y contra un corte fijo (29/10/2026):

- menos de 12 h: PENDIENTE;
- entre 12 h y 1 día: EN_PREPARACION;
- entre 1 y 3 días: ENVIADO;
- más antiguos: ENTREGADO, salvo uno de cada veinte (`id % 20 = 0`), que queda CANCELADO.

Un test lo verifica.

Subconjunto migrado: 9 pedidos existentes

Regla (`sql/01_seleccion_pedidos.sql`, sobre el lote original `id <= 200001`): por cada estado, el pedido de menor id que contiene el producto 13896 y el de menor id que no lo contiene. Así quedan los 5 estados, pedidos con y sin el producto y totales variados para las consultas. No hizo falta agregar pedidos de prueba.

_id	Estado	Con 13896	Líneas	Total
1	PENDIENTE	sí	5	24 782,73
2	ENTREGADO	sí	3	14 249,69
7	ENTREGADO	no	3	14 148,98
20	CANCELADO	sí	4	21 210,34
26	ENVIADO	sí	4	22 611,17
120	CANCELADO	no	3	16 136,83
217	ENVIADO	no	2	8 491,21
240	EN_PREPARACION	sí	4	19 782,73
10167	EN_PREPARACION	no	3	13 918,54

Esquema de la colección `foodstore.pedidos`

```
{ _id: 2,                  // pedido.id
  fecha: ISODate("2026-04-10T03:00:00Z"),
  estado: "ENTREGADO",
  forma_pago: "TRANSFERENCIA",
  cliente: {               // objeto embebido
    id: 4812,              // cliente.id
    nombre: "Usuario4809",
    apellido: "Apellido4809",
    email: "usuario4809@test.com" },
  lineas: [                // arreglo embebido
    { producto: {          //   (pedido_detalle)
        id: 9443,          // producto.id
        nombre: "Producto 9438",
        categoria: "Bebidas" },
      cantidad: 3,
      precio_unitario:     // precio histórico
        Decimal128("2218.29"),
      subtotal: Decimal128("6654.87") },
    ... ],
  total: Decimal128("14249.69") }  // Σ subtotales
```

Los importes van en `Decimal128`, con aritmética decimal exacta como `NUMERIC(10,2)`. Un `double` introduciría errores de redondeo. Las fechas `timestamptz` se guardan en UTC.

Migración (Consigna 2)

`mongodb/01_crear_coleccion.js` crea la base, la colección, el validador y los índices (`estado`, `lineas.producto.id` y `total`). El `insertMany()` de los 9 pedidos y sus 31 líneas lo **genera un script SQL desde PostgreSQL** (`migracion/generar_insert_pedidos.sql`). Así los valores se copian de la base y no se transcriben a mano. El test `tests/test_migracion.sh` compara los dos motores **campo por campo en cada línea**: estado, fecha, forma de pago, cliente, producto, cantidad, precio, subtotal y total.

```
> db.pedidos.countDocuments()                          9
> db.pedidos.getIndexes()   _id_, idx_estado {"estado":1}, idx_lineas_producto {"lineas.producto.id":1}, idx_total
{"total":1}
> opciones del validador    { validationLevel: 'strict', validationAction: 'error', clausulas_en_validator: 5 }
> insert de un pedido con cantidad 0   →   rechazado: code 121 - Document failed validation
$ bash tests/test_migracion.sh   →   test_migracion: OK
```

Reglas de integridad conservadas y qué se embebe (Consigna 3)

1. **Precio histórico, coherente con el total.** `precio_unitario` se copia de `pedido_detalle` y nunca de `producto.precio`, así que un cambio de precio no altera los pedidos ya hechos. En MongoDB, el validador (`$jsonSchema` + `$expr`) exige:
   - `cantidad >= 1`;
   - importes entre 0 y 99 999 999,99, con el precio en 2 decimales;
   - `subtotal = cantidad × precio_unitario`;
   - `total = Σ subtotales`.

   Límite: en estos datos el precio histórico coincide con el actual en las 621 853 líneas, así que la regla la garantiza la forma de migrar, no un caso de prueba.
2. **Identidad y referencias.** `_id = pedido.id` impide duplicar un pedido. Se conservan `cliente.id` y `producto.id`. El validador impide repetir un producto dentro de un pedido, igual que la PK `(pedido_id, producto_id)`, y exige al menos una línea y estados válidos. Límite: MongoDB no tiene FK, así que la existencia de cliente y producto queda garantizada en PostgreSQL.

Cómo se probó. El test verifica que el validador rechace 10 documentos inválidos, cada uno por su regla, según qué cláusula falló en `errInfo.details`. Se probó con 8 mutaciones: quitar una regla o alterar un dato. Todas hicieron fallar el test.

Qué se embebe:

- **Las líneas.** Pertenecen a un único pedido y son pocas (de 1 a 5). No cambian después de cerrado el pedido, y el acceso frecuente es el pedido completo: una lectura, sin el JOIN de tres tablas. Es la relación "contiene" uno a pocos que la documentación de MongoDB recomienda embeber.
- **Una copia de los datos esenciales del cliente y del producto, con su id.** Es una foto del momento de la venta, como el precio histórico.
- **Qué no se embebe.** El resto (teléfono, stock, precio actual) cambia independientemente de los pedidos y se queda en PostgreSQL. Copiarlo es costoso: en estos datos, los 200 000 pedidos generados son del mismo cliente (4812).

2. Consultas en MongoDB (Consigna 4)

	Consulta	Instrucción (`consultas_mongodb.js`)	Resultado (`_id`)
a	Todos, con id y estado	`db.pedidos.find({}, {_id: 1, estado: 1})`	9: 1, 2, 7, 20, 26, 120, 217, 240, 10167
b	Por estado	`db.pedidos.find({estado: "ENTREGADO"})`	2: 2, 7
c	Producto en el arreglo	`db.pedidos.find({"lineas.producto.id": 13896}, {estado: 1, "lineas.$": 1})`	5: 1, 2, 20, 26, 240
d	Total > umbral	`db.pedidos.find({total: {$gt: NumberDecimal("20000.00")}})`	3: 1, 26, 20

- **c)** `"lineas.producto.id"` recorre todos los elementos del arreglo, y `"lineas.$"` muestra solo la línea que coincidió. `explain()` confirma que usa el índice sobre el arreglo: `FETCH ← IXSCAN(idx_lineas_producto, multikey=true)`, con 5 documentos examinados.
- **d) Umbral de 20 000.** En los 200 001 pedidos de la base, la mediana del total es 15 653,28, el tercer cuartil 19 731,93 y el p90 22 510,46. 20 000 queda apenas por encima del tercer cuartil, así que marca los **pedidos grandes** (el 23,9 % del total), con una cifra redonda. La comparación es en `Decimal128` y estricta: el pedido 240 (19 782,73) queda afuera.

```
== c) Pedidos que contienen el producto 13896
{"_id":1,"estado":"PENDIENTE","lineas":[{"producto":{"id":13896,"nombre":"Producto
13891","categoria":"Bebidas"},"cantidad":1,"precio_unitario":{"$numberDecimal":"509.44"},...}]}
{"_id":2,"estado":"ENTREGADO","lineas":[{"producto":{"id":13896,...},"cantidad":2,"precio_unitario":
{"$numberDecimal":"509.44"},"subtotal":{"$numberDecimal":"1018.88"}}]}
... (20, 26, 240)
plan: FETCH <- IXSCAN(idx_lineas_producto, multikey=true) | documentos examinados: 5
RESULTADO c: 1,2,20,26,240
== d) Pedidos con total > 20000.00
{"_id":1,"estado":"PENDIENTE","total":{"$numberDecimal":"24782.73"}}
{"_id":26,"estado":"ENVIADO","total":{"$numberDecimal":"22611.17"}}
{"_id":20,"estado":"CANCELADO","total":{"$numberDecimal":"21210.34"}}
RESULTADO d: 1,20,26
```

3. Equivalentes en PostgreSQL y verificación (Consigna 5)

```
-- c) pedidos migrados con el producto 13896
SELECT p.id, p.estado, d.producto_id, pr.nombre,
       d.cantidad, d.precio_unitario
FROM pedido p
JOIN migrados m USING (id)      -- los 9 pedidos
JOIN pedido_detalle d ON d.pedido_id = p.id
JOIN producto pr ON pr.id = d.producto_id
WHERE d.producto_id = 13896;
-- b) ... WHERE p.estado = 'ENTREGADO'
-- d) total = SUM(subtotal) > 20000.00
```

```
 id  |     estado     | producto_id | cantidad | precio
-----+----------------+-------------+----------+--------
   1 | PENDIENTE      |       13896 |        1 | 509.44
   2 | ENTREGADO      |       13896 |        2 | 509.44
  20 | CANCELADO      |       13896 |        2 | 509.44
  26 | ENVIADO        |       13896 |        4 | 509.44
 240 | EN_PREPARACION |       13896 |        1 | 509.44
RESULTADO c: 1,2,20,26,240
```

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los ids?
Estado seleccionado: `ENTREGADO`	2 (2, 7)	2 (2, 7)	Sí
Producto seleccionado: 13896 ("Producto 13891")	5 (1, 2, 20, 26, 240)	5 (1, 2, 20, 26, 240)	Sí
Adicional: total > 20 000	3 (1, 20, 26)	3 (1, 20, 26)	Sí

La comparación es automática. `tests/test_consultas.sh` ejecuta los dos archivos y compara las líneas `RESULTADO`, que salen de las mismas consultas. Se probó con 5 mutaciones (otro estado, otro umbral, sin la restricción a los pedidos migrados, otro producto, un id de más en la lista) y las 5 lo hicieron fallar.

Diferencias detectadas y cómo se resolvieron:

1. **Volumen: se corrigió la consulta.** La primera versión en PostgreSQL recorría toda la tabla y devolvía 188 745 pedidos `ENTREGADO` y 155 679 con el producto, contra 2 y 5 en MongoDB, que tiene solo el subconjunto migrado. La corrección fue limitar las consultas a los 9 pedidos migrados (la vista `migrados`). Las dos versiones quedaron en el archivo.
2. **Fecha: misma información, distinta representación.** `2026-04-10T03:00:00Z` en MongoDB y `2026-04-10 00:00:00-03` en PostgreSQL son el mismo instante.
3. **Total: guardado en un motor, calculado en el otro.** En MongoDB se guarda y en PostgreSQL se calcula con `SUM`. Coinciden porque el validador exige `total = Σ subtotales`.
4. **Forma de la consulta c).** En MongoDB es un filtro sobre el arreglo, sin uniones. En PostgreSQL requiere unir tres tablas, sin `DISTINCT` gracias a la PK.

No hizo falta corregir el modelo.

4. Evaluación de la tecnología con IA (Consignas 6 y 7)

Consulta. Se consultó a un agente de IA aislado (Claude Sonnet), sin acceso al repositorio, a la conversación ni a herramientas. El prompt describía:

- el modelo relacional actual (`pedido`, `pedido_detalle` con PK compuesta y precio histórico, `producto` y `cliente`, con sus CHECK y FK);
- que el acceso más frecuente es "leer pedidos completos";
- que es "indispensable conservar la coherencia de cantidades y precios".

Se le pidió comparar PostgreSQL y MongoDB y recomendar.

Respuesta (síntesis). Recomendó mantener PostgreSQL como fuente de verdad y agregar MongoDB solo como vista de lectura derivada, si apareciera la necesidad. Su argumento: en MongoDB la coherencia de precios "queda en la aplicación y no en la base". El prompt y la respuesta literales están en `docs/ia/`.

Afirmación 1: `$jsonSchema` *"no puede expresar relaciones entre campos como* `subtotal == cantidad × precio` *ni* `total == suma(subtotales)` **", y la coherencia queda *"Solo por código de aplicación"*

Incorrecta en lo que importa. Es cierto que las palabras clave de `$jsonSchema` comparan contra valores fijos. Pero el validador de una colección no se limita a `$jsonSchema`.

- **Documentación oficial** (*Specify Validation With Query Operators*, manual de MongoDB). Los operadores de consulta permiten comparar campos entre sí. El ejemplo del manual es una colección de pedidos que exige `totalWithVAT = total * (1 + VAT)`, y esa validación se combina con JSON Schema.
- **Laboratorio.** El validador de `foodstore.pedidos` rechazó con el error 121 tres escrituras:
  - un subtotal distinto de 3 × 2218,29;
  - un total distinto de la suma de los subtotales;
  - un `updateOne` que intentaba desincronizar el total del pedido 2, que se quedó en 14 249,69.
- **Lo que la IA sí acierta.** MongoDB no tiene claves foráneas.
- **Lo que agrega el equipo.** El validador controla cada documento por separado y puede saltearse con `bypassDocumentValidation`.

Afirmación 2: "*Las transacciones multi-documento existen desde la 4.0, pero son más costosas*". La transacción "pedido + stock" figura como "Posible"

Correcta pero incompleta. La omisión afecta justo al laboratorio.

- **Documentación oficial** (*Production Considerations* y *Transactions*):
  - exige una compatibilidad mínima 4.0 para conjuntos de réplicas y 4.2 para clústeres fragmentados;
  - aclara que una instalación **standalone no admite transacciones**;
  - confirma que una transacción suele costar más que escribir un solo documento y que un modelo desnormalizado suele ser el óptimo.
- **Laboratorio.** El MongoDB del TP es standalone (`hello()` sin `setName`).

```
> mongosh: session.startTransaction() + updateOne + commit
  rechazada (driver): This MongoDB deployment does not support retryable writes.
> comando al servidor: update con lsid + txnNumber + startTransaction
  rechazado por el servidor: code 20 (IllegalOperation) - Transaction numbers are only allowed on a replica set member
or mongos
```

Grabar el pedido no necesita transacción, porque la operación sobre un documento es atómica. "Pedido + stock" sí la necesitaría, y habría que montar un conjunto de réplicas.

Una afirmación más que se confirmó. Según la IA, "*el diferencial de rendimiento de MongoDB en la lectura (evitar 3 JOIN indexados) es pequeño*". Para leer el pedido 2 completo:

- PostgreSQL ejecutó la consulta en 0,32 ms, con 5 índices y 25 bloques leídos;
- MongoDB usó `EXPRESS_IXSCAN`: 1 clave y 1 documento, en menos de 1 ms.

Conclusión (199 palabras)

Conviene **mantener PostgreSQL como fuente de verdad de los pedidos** y usar MongoDB, si hiciera falta, solo como copia de lectura derivada de PostgreSQL. Se llega a la misma recomendación que la IA, pero por razones corregidas.

Primer criterio: integridad y atomicidad. MongoDB pudo validar dentro de cada documento lo que la IA creía imposible: `subtotal = cantidad × precio`, `total = Σ subtotales` y producto sin repetir. Lo que no puede hacer es garantizar las relaciones entre documentos. No tiene claves foráneas, y la operación conjunta "pedido + stock" requiere transacciones, que el MongoDB standalone del laboratorio rechazó (código 20). PostgreSQL ofrece las dos cosas en cualquier instalación.

Segundo criterio: patrón de acceso. El documento embebido resuelve el pedido completo con una sola lectura. Pero PostgreSQL lo hizo en 0,32 ms usando cinco índices. A esta escala, la ventaja no justifica un segundo motor.

Tercer criterio: duplicación. Embeber al cliente copia sus datos en cada pedido. Como foto histórica sirve, pero cualquier dato que deba estar al día obligaría a reescribir documentos: el cliente 4812 tiene 200 000 pedidos. En PostgreSQL es una fila.

La combinación solo se justificaría con mucho tráfico de lectura de pedidos cerrados, que no cambian.